



Sumario

UT1: Preparación del entorno.....	2
1 Pasos para la preparación del entorno.....	2
2 Programa de prueba.....	7



Documentación oficial de **Python**: <https://docs.python.org/3/>

Referencia **VSCode** : https://code.visualstudio.com/docs/python/python-tutorial#_prerequisites



Actualización: Septiembre 2024

Realizado bajo licencia [Attribution-NonCommercial-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-nc-sa/4.0/)
Vanesa García López (vgarcialopez@educa.madrid.org)

Profesora: Julia Gallego Sanchez (julia.gs@educa.madrid.org)



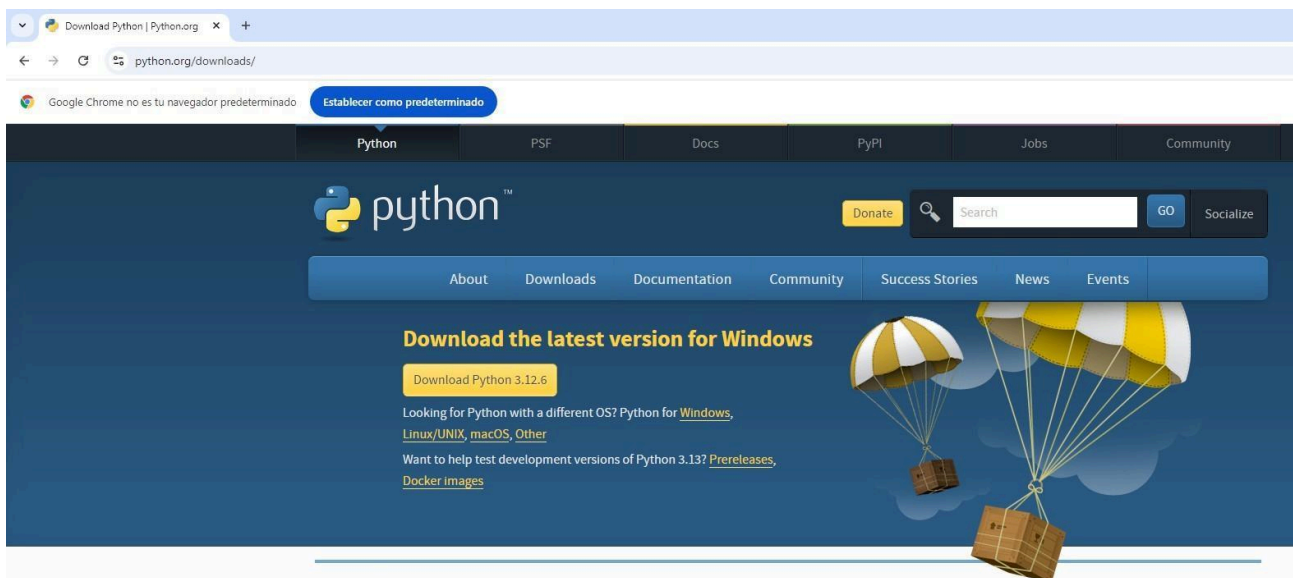
UT1: Preparación del entorno

1 Pasos para la preparación del entorno.

Para realizar la preparación del entorno seguiremos los siguientes pasos:

1. Instalar el intérprete de Python dependiendo del sistema operativo que usemos. En nuestro caso Windows.

<https://www.python.org/downloads/>



2. Proceso de instalación en Windows.

En la ventana de instalación seleccionar las dos opciones:

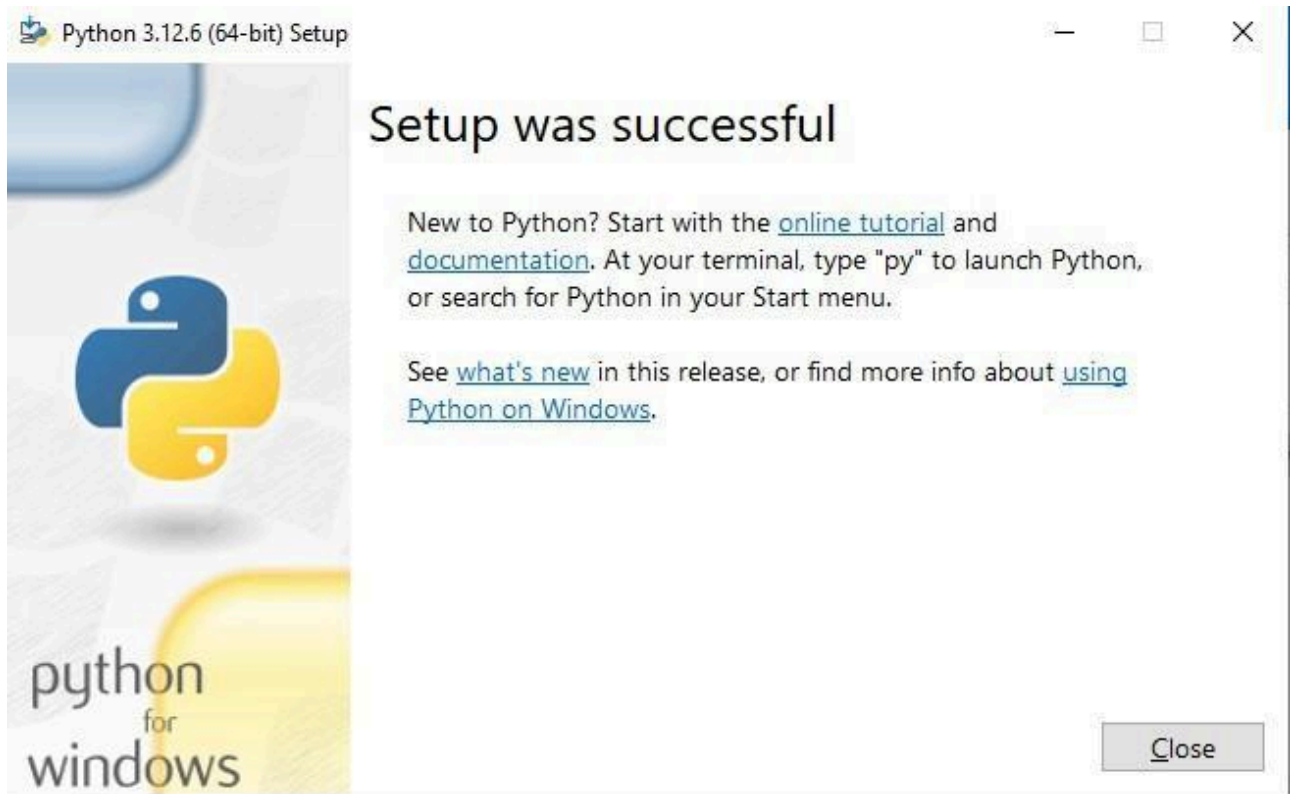
- Use admin privileges when installing py.exe
- Add python.exe to PATH



Comenzar la instalación seleccionando Install Now.

→ **Install Now**
C:\Users\Administrador\AppData\Local\Programs\Python\Python312
Includes IDLE, pip and documentation
Creates shortcuts and file associations

Y esperar a que la instalación termine.



3. Comprobar que la versión instalada se la realizado con éxito en Windows con cmd:

```
py -3 --version
```

Copy

4. Descargar e instalar Visual Studio Code para Windows en caso de no tenerlo instalado ya.

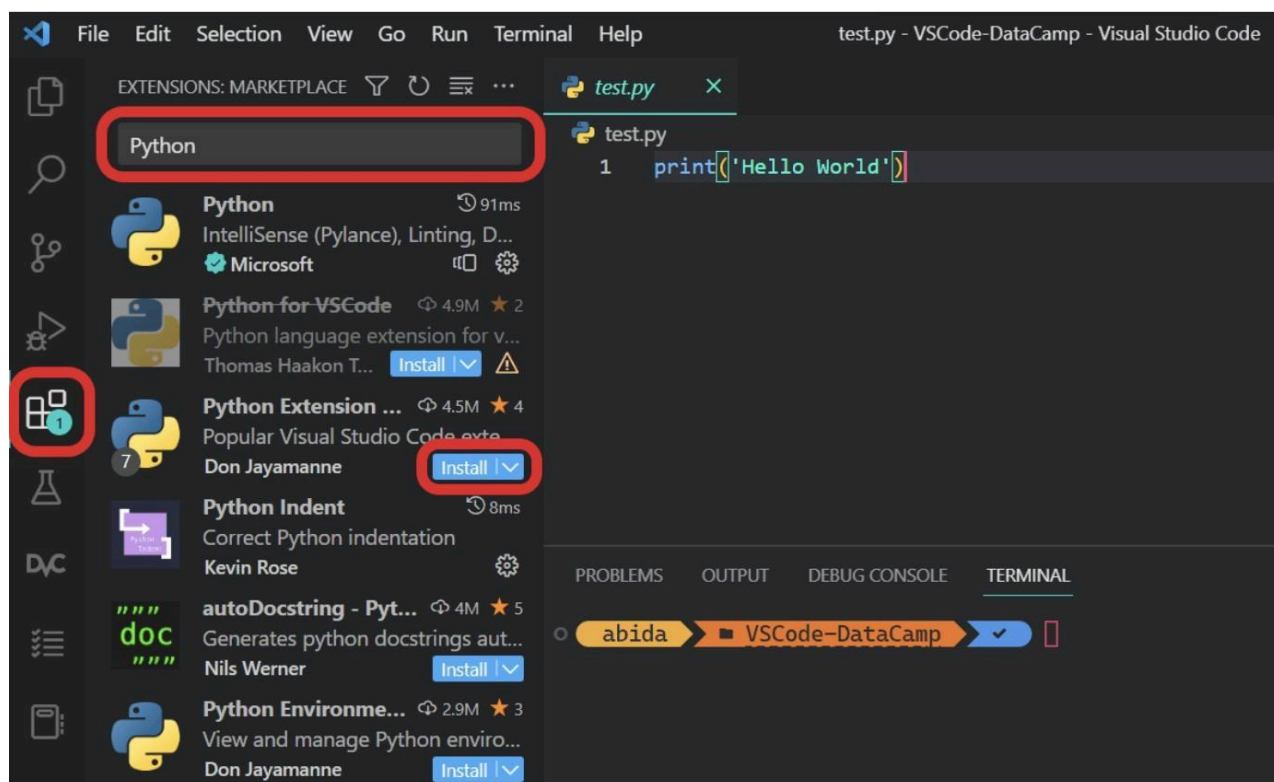
<https://code.visualstudio.com/Download>

Extensiones a instalar en VSCode para trabajar con Python:

EXTENSIÓN	CARACTERÍSTICAS
Phython	La extensión de Python incluye IntelliSense, linting, depuración, navegación por el código, formateo del código, refactorización, explorador de variables y explorador de pruebas. <u>IntelliSense</u> (autocompletar código) <u>Linting</u> (Pylint, Flake8) <u>Formateo de código</u> (black, autopep) <u>Depuración</u> <u>Pruebas</u> (unittest, pytest) <u>Jupyter Notebooks</u> <u>Entornos</u> (venv, pipenv, conda) <u>Refactorización</u>

EXTENSIONES OPCIONALES	
Indent - rainbow	Las extensiones <u>Indent-rainbow</u> nos proporcionan una sangría coloreada multinivel para mejorar la legibilidad del código. Obtenemos colores alternos en cada paso, y nos ayuda a evitar errores comunes de sangría.
Python Indent	La extensión <u>Python Indent</u> nos ayuda a crear sangrías. Al pulsar la tecla Intro, la extensión analizará el archivo Python y determinará cómo debe sangrarse la siguiente línea. Ahorra tiempo.
Jupiter Notebook Renderers	<u>Jupyter Notebook Renderers</u> forma parte del paquete de extensiones <u>Jupyter</u> . Nos ayuda a renderizar salidas plotly, vega, gif, png, svg y jpeg.
autoDocstring	La extensión autoDocstring nos ayuda a generar rápidamente docstrings para las funciones de Python. Escribiendo las comillas triples """ o ''' dentro de la función, podemos generar y modificar docstring. Aprende más sobre las docstrings siguiendo nuestro tutorial <u>Docstrings de Python</u> .

Haz clic en el icono del recuadro de la barra de actividades o utiliza un atajo de teclado: Ctrl + Mayús + X para abrir el panel de extensiones. Escribe cualquier palabra clave en la barra de búsqueda para explorar todo tipo de extensiones.

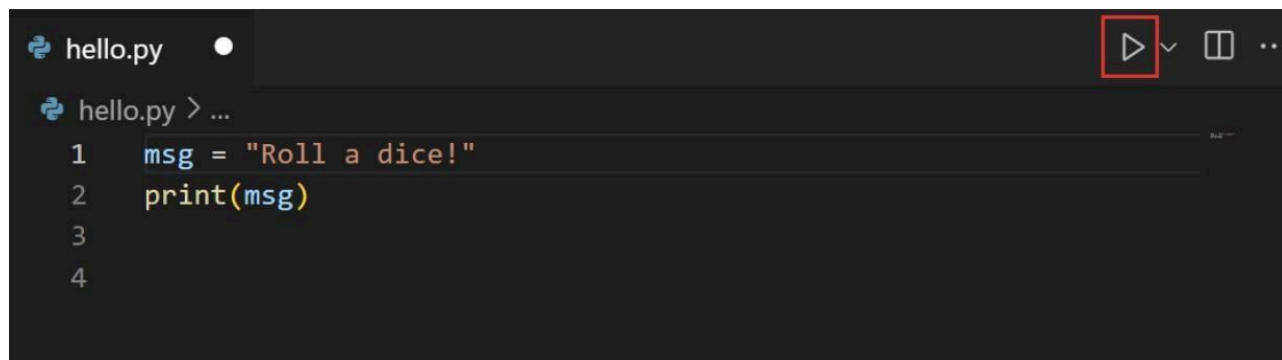


En nuestro caso, escribiremos Python e instalaremos la extensión Python haciendo clic en el botón de instalación, como se muestra arriba.

La extensión [Python](#) instala automáticamente las extensiones [Pylance](#), [Jupyter](#) e [isort](#). Incluye una completa colección de herramientas para ciencia de datos, desarrollo web e ingeniería de software.

2 Programa de prueba

Crearemos un programa de prueba Hola mundo creando el fichero con extensión .py



```
hello.py
1 msg = "Roll a dice!"
2 print(msg)
3
4
```

Ejecutaremos el programa en el Terminal pulsando el botón situado en la parte superior derecha del editor.

El botón abrirá un panel de terminal en el que se activa automáticamente el intérprete de Python ejecutando `python hello.py` en Windows:

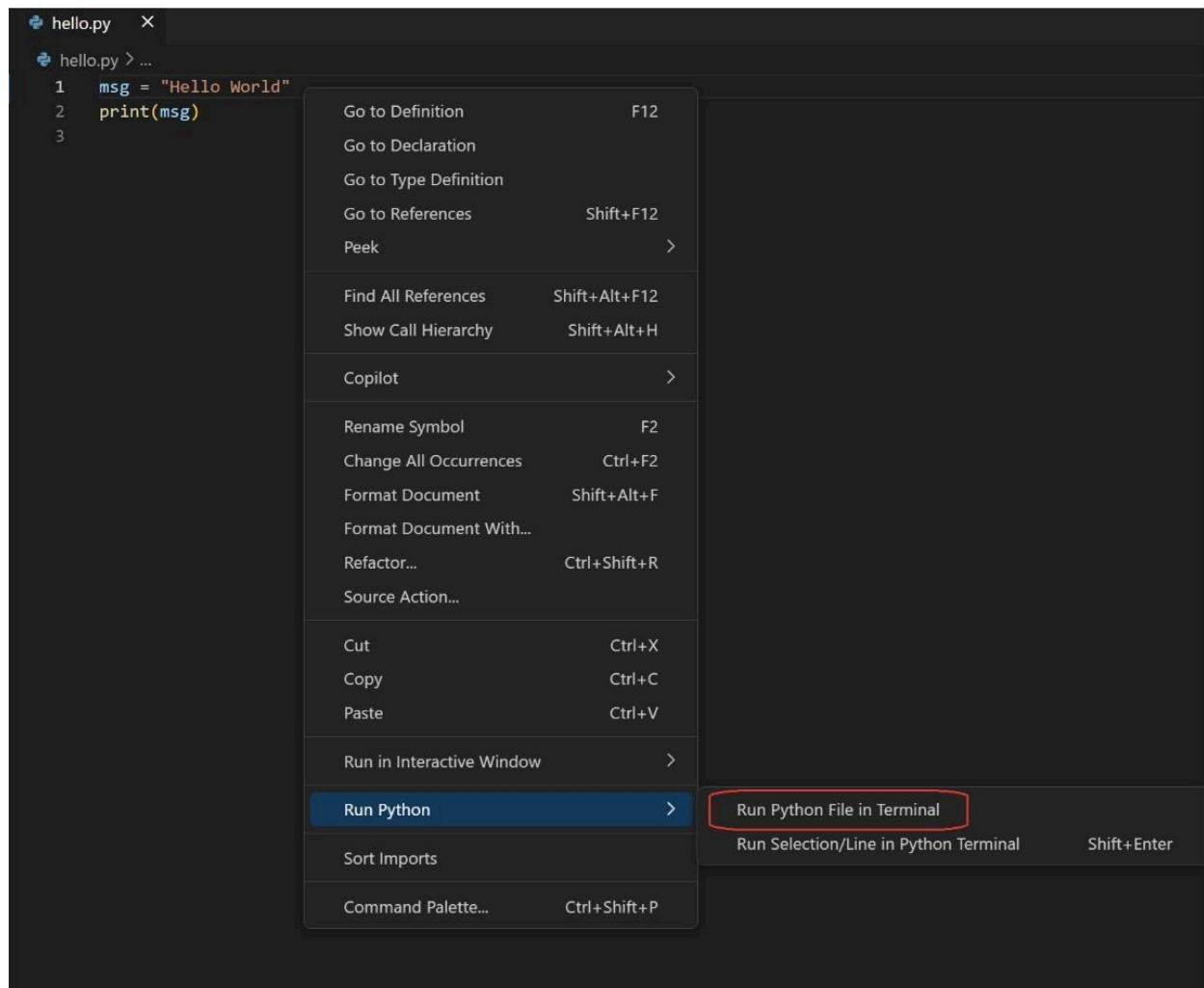


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Python + - [ ] [X] ... ^ X

PS C:\hello> & c:/hello/.venv/Scripts/python.exe c:/hello/hello.py
Roll a dice!
PS C:\hello>
```

Existen otras formas de ejecutar Python en VSCode:

- Haciendo clic derecho en cualquier lugar de la ventana del editor y seleccione Ejecutar > Archivo Python en la Terminal (que guarda el archivo automáticamente):



- Seleccionando una o más líneas, luego presionando Shift+Enter o haciendo clic derecho y seleccionando Ejecutar selección/línea en la terminal Python. Este comando es conveniente para probar solo una parte de un archivo.

3 Desarrollar proyectos en Python

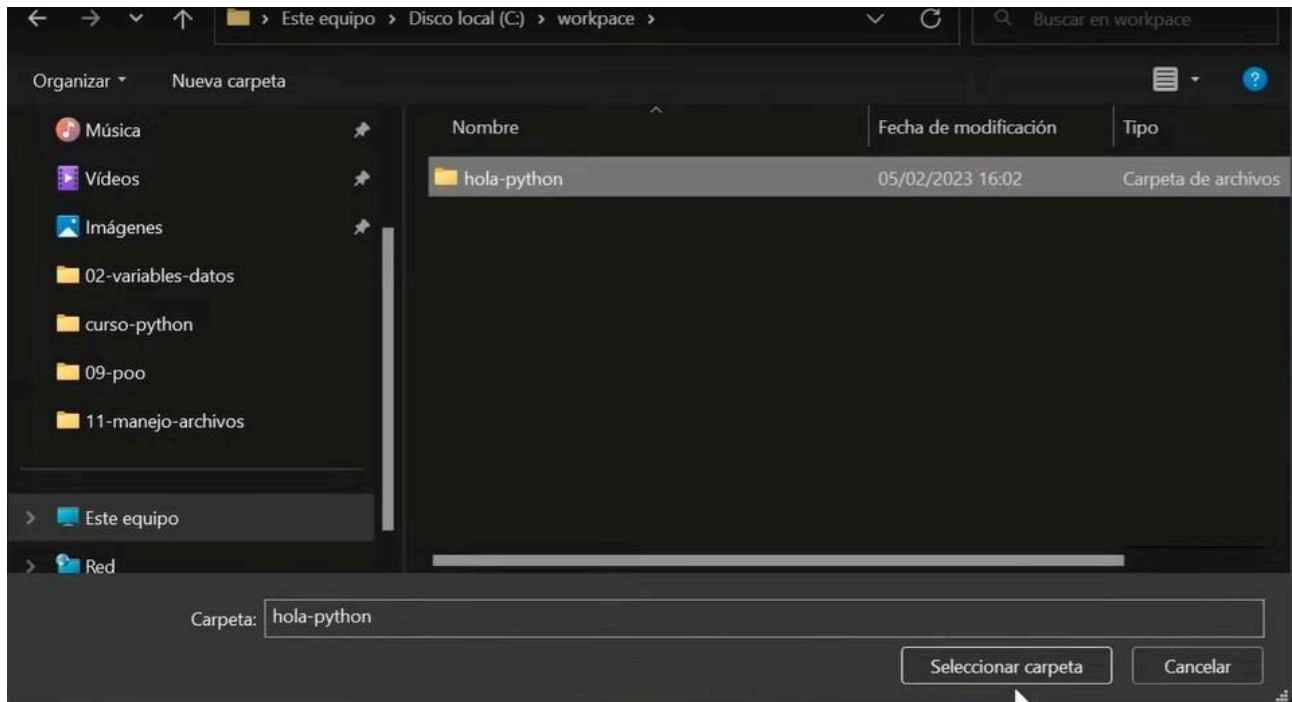
En la documentación oficial de Visual Studio Code podrás encontrar guías de cómo trabaja el IDE con Python y otros lenguajes de programación:

<https://code.visualstudio.com/docs/python/python-quick-start>

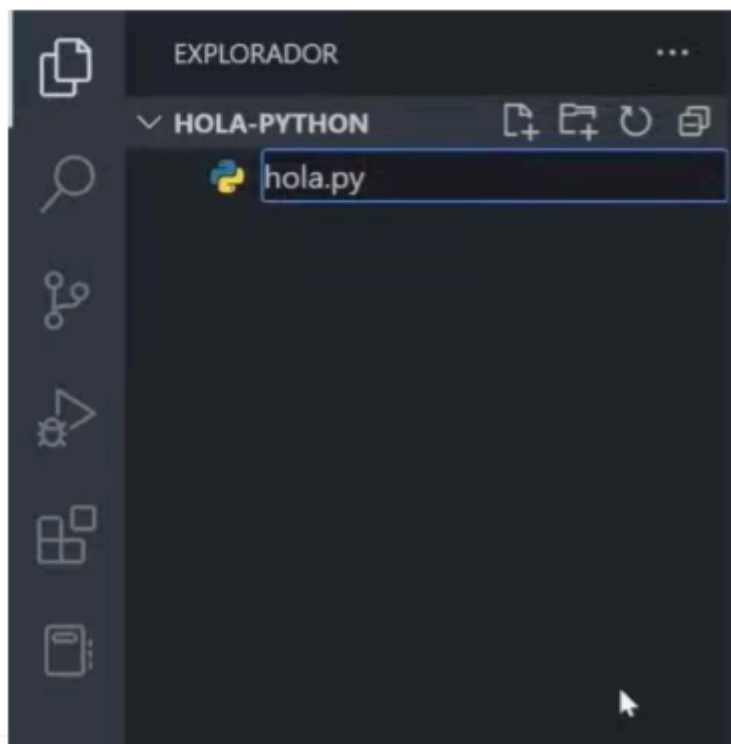
Existen diferentes formas de trabajar en proyectos de desarrollo con Visual Studio Code:

1. Open File (Abrir archivo): Podemos crear un nuevo archivo Python seleccionando New File en la página de bienvenida de Visual Studio Code y luego seleccionando el archivo Python, o navegando en File > New File
2. Open Folder (Abrir carpeta):
Podemos crear nuevas carpetas y archivos utilizando los íconos New Folders o New Files correspondientes en la carpeta de nivel superior en la vista del Explorador de archivos una vez seleccionado la carpeta raíz de nuestro proyecto.

Crearemos una carpeta en nuestro espacio de trabajo, por ejemplo con el nombre workspace y dentro de ella una carpeta para nuestro nuevo proyecto, por ejemplo hola-python:



En Visual Studio Code ya sólo tendremos que seleccionar la carpeta workspace.



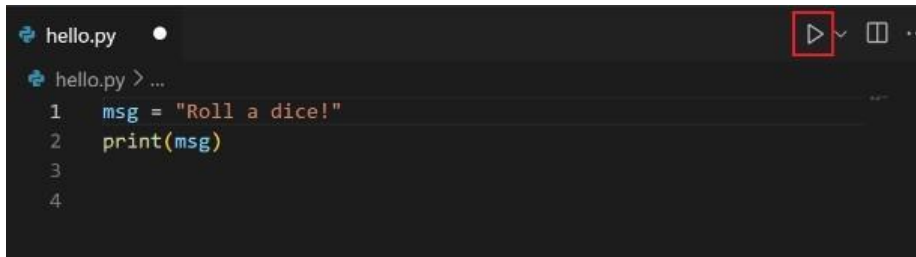
Desde el explorador podremos:

- Crear archivos con extensión .py
- Crear carpetas para organizar los archivos de nuestro proyecto.



Con Visual Studio Code y la extensión de Python podremos:

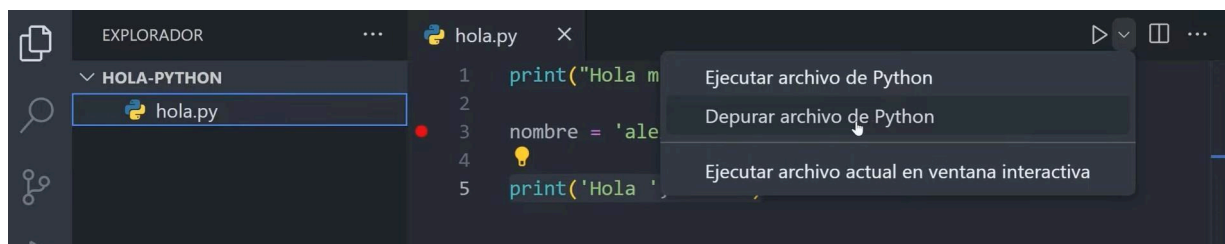
1. Run: Ejecutar programas



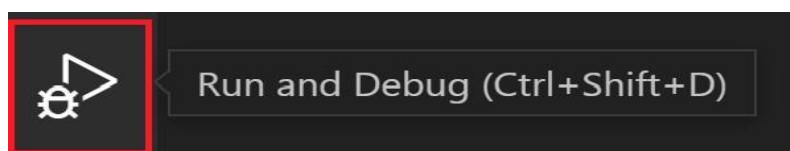
2. Debug: Depurar programas o “debuguear”

El depurador es una herramienta útil que permite inspeccionar el flujo de ejecución del código e identificar errores más fácilmente, así como explorar cómo cambian las variables y datos a medida que se ejecuta el programa.

Para comenzar a depurar hay que establecer puntos de interrupción haciendo clic en el canal al lado de la línea a inspeccionar.



Para iniciar la depuración, hay que pulsar F5 o bien el icono:



Al ser la primera vez que se depura este archivo, se abrirá un menú de configuración que le permitirá seleccionar el tipo de aplicación que desea depurar. Si es un script de Python, puede seleccionar archivo Python.

Una vez que el programa alcanza el punto de interrupción, se detendrá y podremos realizar un seguimiento de los datos en la consola de depuración de Python y avanzar en el programa utilizando la barra de herramientas de depuración.



Los botones de izquierda a derecha, representan las siguientes acciones:

- **Continue:** Reanuda la ejecución del programa hasta el siguiente *breakpoint* (punto de interrupción) o hasta que el programa finalice.
- **Step Over:** Avanza a la siguiente línea de código, pero si esa línea contiene una función, no entra en ella, sino que la ejecuta por completo antes de avanzar.
- **Step Into:** Avanza a la siguiente línea de código y, si esa línea contiene una función, entra en ella para depurarla paso a paso.
- **Step Out:** Si te encuentras dentro de una función, ejecuta el resto de la función y regresa al nivel superior (la línea que llamó a la función).

- **Restart:** Reinicia el programa desde el principio, aplicando de nuevo los puntos de interrupción con figurados.
- **Stop:** Detiene la ejecución del programa y termina la sesión de depuración.

3. Test: el testing es el proceso de evaluar y verificar que un programa o sistema funciona correctamente y cumple con los requisitos esperados.

Se pueden realizar pruebas unitarias que verifiquen un funciones específicas de código o bien utilizar un framework de pruebas como [Unittest](#) y [pytest](#) disponible en la extensión Phyton.